

Umum

- **Regresi:** output angka kontinu. Contoh: harga, suhu, energi.
- **Klasifikasi:** output kelas. Contoh: spam/tidak, penyakit/tidak.
- **Clustering:** data tanpa label, cari kelompok.
- **PCA:** reduksi dimensi/kompresi fitur.
- **Anomaly detection:** cari data aneh/jarang.
- **RL/Q-learning:** agent belajar aksi dari reward.

Checklist menjawab soal:

1. Tentukan jenis masalah: regresi, klasifikasi, clustering, PCA, anomaly, atau RL.
2. Tulis notasi: $m, n, x^{(i)}, y^{(i)}, \theta, w, b$.
3. Pilih rumus model: linear, logistic, NN, SVM, K-means, PCA, Gaussian, Q-learning.
4. Hitung cost/metric/update sesuai pertanyaan.
5. Jelaskan interpretasi: underfit/overfit, threshold, margin, cluster, variance, anomaly, reward.

Paradigma Pembelajaran

Supervised Learning: data punya label/jawaban benar.

$$D = \{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}.$$

Tujuan: belajar mapping $x \mapsto y$. Contoh: linear/logistic regression, SVM, neural network.

Unsupervised Learning: data tidak punya label.

$$D = \{x^{(1)}, x^{(2)}, \dots, x^{(m)}\}.$$

Tujuan: mencari struktur/pola. Contoh: K-means dan PCA.

Reinforcement Learning (RL): agent berinteraksi dengan environment, memilih action, menerima reward, dan belajar memaksimalkan reward kumulatif.

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

Komponen: state s , action a , reward r , policy π , discount γ , environment, agent.

Jenis	Inti Perbedaan
Supervised	Ada label y ; belajar prediksi $x \rightarrow y$.
Unsupervised	Tanpa label; cari cluster/kompresi/struktur.
RL	Tidak diberi label benar langsung; belajar dari reward.

Notasi Dasar

Simbol	Arti
m	jumlah training examples
n	jumlah fitur/features
$x^{(i)}$	input training example ke- i
$x_j^{(i)}$	fitur ke- j pada data ke- i
$y^{(i)}$	target/label data ke- i
θ, w, b	parameter/bobot/bias model
$h_{\theta}(x)$	hypothesis/prediksi model
X	matriks data, biasanya $m \times (n + 1)$ jika ada $x_0 = 1$

$$\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}, \quad x^{(i)} \in \mathbb{R}^{n+1}, \quad x_0^{(i)} = 1.$$

Bias/intercept biasanya dimasukkan sebagai $x_0 = 1$. Regularisasi biasanya **tidak** menghukum θ_0 .

Aljabar Linear Cepat

Matriks $A \in \mathbb{R}^{m \times n}$ punya m baris dan n kolom. Vektor kolom $x \in \mathbb{R}^n$ dianggap $n \times 1$.

$$A_{m \times n} B_{n \times p} = C_{m \times p}, \quad C_{ij} = \sum_{k=1}^n A_{ik} B_{kj}.$$

$$(AB)^T = B^T A^T, \quad y = \hat{y} = h_{\theta}(X) = X\theta.$$

Jika X berukuran $m \times (n + 1)$ dan θ berukuran $(n + 1) \times 1$, maka $X\theta$ berisi semua prediksi untuk m data.

Linear Regression

Dipakai untuk memprediksi output kontinu.

Model

Satu variabel:

$$h_{\theta}(x) = \theta_0 + \theta_1 x.$$

Banyak fitur:

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n = \theta^T x.$$

Cost Function / MSE

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)}\right)^2.$$

Bentuk vektor:

$$J(\theta) = \frac{1}{2m} \|X\theta - y\|_2^2.$$

Faktor $\frac{1}{2}$ membuat turunan lebih sederhana.

Gradient Descent

Update semua θ_j secara simultan:

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)}\right) x_j^{(i)}.$$

Bentuk vektor:

$$\theta := \theta - \alpha \frac{1}{m} X^T (X\theta - y).$$

α = learning rate. Terlalu besar: cost bisa naik/divergen. Terlalu kecil: konvergensi lambat.

Normal Equation

$$\theta = (X^T X)^{-1} X^T y.$$

Tidak perlu memilih α , tetapi mahal untuk n besar dan bermasalah jika $X^T X$ non-invertible.

Multiple Features, Scaling, Polynomial

Feature scaling: buat fitur berada pada skala mirip agar gradient descent lebih cepat.

$$x_j \leftarrow \frac{x_j - \mu_j}{s_j}.$$

μ_j = rata-rata fitur j ; s_j = range atau standar deviasi.

Polynomial features: model linear dapat menangkap pola non-linear dengan fitur baru.

$$h_{\theta}(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3.$$

Catatan: fitur polinomial rawan overfitting, sering perlu regularisasi.

Istilah	Arti Cepat
Feature	variabel input model
Target	nilai yang ingin diprediksi
Parameter	bobot yang dipelajari model
Hyperparameter	pengaturan sebelum training, misalnya α, λ, K
Epoch	satu kali proses melewati seluruh data training

Logistic Regression dan Klasifikasi

Walau namanya regression, logistic regression dipakai untuk **klasifikasi**.

Sigmoid / Logistic Function

$$g(z) = \frac{1}{1 + e^{-z}}, \quad h_{\theta}(x) = g(\theta^T x).$$

Interpretasi:

$$h_{\theta}(x) = P(y = 1 \mid x; \theta), \quad P(y = 0 \mid x; \theta) + P(y = 1 \mid x; \theta) = 1.$$

Decision Rule

$$h_{\theta}(x) \geq 0.5 \Rightarrow \hat{y} = 1, \quad h_{\theta}(x) < 0.5 \Rightarrow \hat{y} = 0.$$

Karena $g(z) \geq 0.5$ ketika $z \geq 0$:

$$\theta^T x \geq 0 \Rightarrow \hat{y} = 1, \quad \theta^T x < 0 \Rightarrow \hat{y} = 0.$$

Decision boundary adalah garis/kurva yang memisahkan kelas.

Cost Function Logistic Regression

Untuk satu data:

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log h_{\theta}(x), & y = 1, \\ -\log(1 - h_{\theta}(x)), & y = 0. \end{cases}$$

Bentuk gabungan:

$$\text{Cost}(h_{\theta}(x), y) = -y \log h_{\theta}(x) - (1 - y) \log(1 - h_{\theta}(x)).$$

Semua data:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right].$$

Gradient descent bentuknya sama dengan linear regression, tetapi h_{θ} memakai sigmoid.

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)}\right) x_j^{(i)}.$$

Multi-class / One-vs-All

Latih satu classifier untuk setiap kelas.

y-hat = arg max_k h_theta^{(k)}(x).

Evaluasi Model dan Confusion Matrix

Table with 3 columns: , Prediksi 1, Prediksi 0. Rows: Aktual 1, Aktual 0.

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Error Rate = (FP + FN) / (TP + TN + FP + FN)

Precision = TP / (TP + FP), Recall = TP / (TP + FN)

Specificity = TN / (TN + FP), F1 = (2 * Precision * Recall) / (Precision + Recall)

Kapan dipakai:

- **Accuracy:** kelas relatif seimbang.
- **Precision:** false positive mahal. Contoh: email normal jangan ditandai spam.
- **Recall:** false negative mahal. Contoh: penyakit/anomali jangan terlewat.

Training set: melatih parameter. Validation/CV set: memilih model/hyperparameter. Test set: estimasi kinerja akhir pada data baru.

Bias, Variance, Regularization

Table with 3 columns: Kondisi, Training error, CV/Test error. Rows: High bias / underfit, High variance / overfit, Good fit.

Regularization menghukum parameter besar agar model tidak terlalu kompleks. Biasanya theta_0 tidak ikut dihukum.

Regularized linear regression:

J(theta) = 1/(2m) * sum_{i=1}^m (h_theta(x^{(i)}) - y^{(i)})^2 + lambda/(2m) * sum_{j=1}^n theta_j^2.

Regularized logistic regression:

J(theta) = -1/m * sum_{i=1}^m [y^{(i)} log h_theta(x^{(i)}) + (1 - y^{(i)}) log (1 - h_theta(x^{(i)}))] + lambda/(2m) * sum_{j=1}^n theta_j^2.

Update untuk j = 0 tanpa regularisasi:

theta_0 := theta_0 - alpha * 1/m * sum_{i=1}^m (h_theta(x^{(i)}) - y^{(i)}) * x_0^{(i)}.

Update untuk j >= 1:

theta_j := theta_j * (1 - alpha * lambda/m) - alpha * 1/m * sum_{i=1}^m (h_theta(x^{(i)}) - y^{(i)}) * x_j^{(i)}.

Efek lambda: terlalu besar => underfit/high bias; terlalu kecil => overfit/high variance.

Neuron, Perceptron, Linear Associator, ADALINE

Neuron buatan menerima input x_1, ..., x_n, bobot w_1, ..., w_n, dan bias b.

z = w^T x + b = sum_{j=1}^n w_j x_j + b, y-hat = phi(z).

Perceptron adalah neuron linear dengan aktivasi threshold/hard-limit, cocok untuk klasifikasi biner yang linearly separable.

y-hat = phi(w^T x + b), phi(z) = { 1, z >= 0; 0, z < 0. }

Aturan update:

w <- w + eta(y - y-hat)x, b <- b + eta(y - y-hat).

Jika data tidak linearly separable, perceptron dapat gagal konvergen.

Linear Associator adalah jaringan linear satu lapis untuk memetakan pola input ke pola output.

y-hat = Wx.

Untuk pasangan asosiasi (x^{(p)}, y^{(p)}):

W = sum_{p=1}^P y^{(p)} (x^{(p)})^T.

Intuisi: associative memory, yaitu menyimpan hubungan input-output.

ADALINE atau Adaptive Linear Neuron mirip perceptron, tetapi belajar dari output linear sebelum threshold.

y_in = w^T x + b, e = d - y_in.

Fungsi biaya satu data:

E = 1/2 (d - y_in)^2.

Aturan LMS/delta rule:

w <- w + eta ex, b <- b + eta e.

Keputusan kelas tetap bisa memakai threshold:

y-hat = { 1, y_in >= 0; 0, y_in < 0. }

Table with 2 columns: Model, Ciri Utama. Rows: Perceptron, Linear Associator, ADALINE.

Neural Network: Representasi

Struktur:

- Layer 1 = input layer.
- Layer tengah = hidden layer.
- Layer terakhir = output layer.
- s_l = jumlah unit di layer l tanpa bias.
- Theta^{(l)} menghubungkan layer l ke layer l + 1.

Theta^{(l)} in R^{s_{l+1} x (s_l + 1)}.

Kolom tambahan +1 berasal dari bias unit.

Forward propagation:

a^{(1)} = x, z^{(l+1)} = Theta^{(l)} a^{(l)}, a^{(l+1)} = g(z^{(l+1)}), h_theta(x) = a^{(L)}.

Tambahkan bias a_0^{(l)} = 1 untuk layer yang bukan output.

Output klasifikasi:

- Binary classification: 1 output unit, y in {0, 1}.
- Multi-class K kelas: K output units, label sering one-hot.

y in R^K, y = [0 1 0 ... 0]^T.

NN Cost dan Backpropagation

Cost function multi-class neural network:

J(Theta) = -1/m * sum_{i=1}^m sum_{k=1}^K [y_k^{(i)} log h_theta(x^{(i)})_k + (1 - y_k^{(i)}) log (1 - h_theta(x^{(i)})_k)]

Dengan regularisasi:

J_reg(Theta) = J(Theta) + lambda/(2m) * sum_{l=1}^{L-1} sum_{i=1}^{s_{l+1}} sum_{j=1}^{s_l} (Theta_{ij}^{(l)})^2.

Kolom bias j = 0 tidak ikut regularisasi.

Backpropagation menghitung gradient dari layer output ke belakang. Untuk output layer:

delta^{(L)} = a^{(L)} - y.

Untuk hidden layer:

delta^{(l)} = ((Theta^{(l)})^T delta^{(l+1)}) o g'(z^{(l)}).

Akumulasi gradient:

Delta^{(l)} := Delta^{(l)} + delta^{(l+1)} (a^{(l)})^T, partial J / partial Theta_{ij}^{(l)} = 1/m Delta_{ij}^{(l)}.

Dengan regularisasi untuk j >= 1:

partial J_reg / partial Theta_{ij}^{(l)} = 1/m Delta_{ij}^{(l)} + lambda/m Theta_{ij}^{(l)}.

Random initialization: jangan mulai semua bobot dari nol karena neuron akan belajar fitur yang sama.

$$\Theta_{ij}^{(l)} \sim \text{Uniform}(-\varepsilon, \varepsilon).$$

CNN: Convolutional Neural Network

CNN memanfaatkan struktur spasial gambar. Filter kecil digeser pada gambar untuk mengekstrak fitur lokal.

Linear score dan 2-layer network:

$$f(x) = Wx, \qquad f(x) = W_2 \max(0, W_1x).$$

max(0, z) adalah ReLU.

Chain rule untuk backprop. Jika $z = f(x, y)$ dan loss L :

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial x}, \qquad \frac{\partial L}{\partial y} = \frac{\partial L}{\partial z} \frac{\partial z}{\partial y}.$$

Convolution sederhana:

$$(I * K)_{i,j} = \sum_u \sum_v I_{i+u,j+v} K_{u,v}.$$

Ukuran output convolution/pooling. Untuk input $H \times W$, filter $F_H \times F_W$, padding P , stride S :

$$H_{out} = 1 + \frac{H + 2P - F_H}{S}, \qquad W_{out} = 1 + \frac{W + 2P - F_W}{S}.$$

Jika ada N_F filter dan input punya C channel:

$$\# \text{ param} = N_F(F_H F_W C + 1).$$

+1 untuk bias per filter.

Support Vector Machine (SVM)

SVM mencari decision boundary dengan **margin** besar. Untuk $y = 1$, ingin $\theta^T x$ besar positif; untuk $y = 0$, ingin $\theta^T x$ besar negatif.

Objektif SVM bentuk umum:

$$\min_{\theta} C \sum_{i=1}^m \text{Cost}(\theta^T x^{(i)}, y^{(i)}) + \frac{1}{2} \sum_{j=1}^n \theta_j^2.$$

Interpretasi C :

- C besar: penalti salah klasifikasi besar, margin bisa lebih sempit, rawan overfit.
- C kecil: margin lebih lebar, toleran salah klasifikasi, bisa underfit.

Kernel / Similarity

Gaussian/RBF kernel:

$$f_i = \text{similarity}(x, l^{(i)}) = \exp\left(-\frac{\|x - l^{(i)}\|^2}{2\sigma^2}\right).$$

Jika x dekat landmark $l^{(i)}$, maka $f_i \approx 1$; jika jauh, $f_i \approx 0$.

Parameter	Efek Cepat
C besar	low bias, high variance, boundary lebih mengikuti data
C kecil	high bias, low variance, boundary lebih halus
σ besar	fitur kernel berubah lambat, boundary halus
σ kecil	fitur kernel berubah cepat, boundary kompleks

Clustering dan K-Means

Clustering = unsupervised learning untuk mengelompokkan data tanpa label. K-Means mencari K centroid.

Algoritma:

- Inisialisasi centroid $\mu_1, \dots, \mu_K$.
- Assignment: pilih centroid terdekat untuk setiap data.
- Update: centroid menjadi rata-rata data dalam cluster.
- Ulangi sampai konvergen.

$$c^{(i)} = \arg \min_k \|x^{(i)} - \mu_k\|^2.$$

$$\mu_k = \frac{1}{|C_k|} \sum_{i: c^{(i)}=k} x^{(i)}.$$

Cost/distortion:

$$J = \frac{1}{m} \sum_{i=1}^m \|x^{(i)} - \mu_{c^{(i)}}\|^2.$$

Memilih K : elbow method atau tujuan aplikasi. K-Means sensitif terhadap inisialisasi; jalankan beberapa kali dan pilih cost terkecil.

PCA: Dimensionality Reduction

Tujuan: kompres data, visualisasi, mengurangi fitur, atau mempercepat training. PCA **tidak memakai label** y .

Preprocessing wajib:

- Mean normalization.
- Feature scaling jika skala fitur jauh berbeda.

$$x_j \leftarrow \frac{x_j - \mu_j}{s_j}.$$

Jika baris X adalah data yang sudah dinormalisasi:

$$\Sigma = \frac{1}{m} X^T X, \qquad [U, S, V] = \text{svd}(\Sigma).$$

Ambil $U_{reduce} = k$ kolom pertama dari U .

$$z = U_{reduce}^T x, \qquad x_{approx} = U_{reduce} z.$$

$z \in \mathbb{R}^k$ adalah representasi dimensi rendah.

Memilih k berdasarkan retained variance:

$$\frac{\sum_{i=1}^k S_{ii}}{\sum_{i=1}^n S_{ii}} \geq 0.99 \quad \text{atau} \quad \frac{\frac{1}{m} \sum_i \|x^{(i)} - x_{approx}^{(i)}\|^2}{\frac{1}{m} \sum_i \|x^{(i)}\|^2} \leq 0.01.$$

Anomaly Detection

Tujuan: mendeteksi data yang sangat jarang/tidak biasa berdasarkan density $p(x)$.

Gaussian satu fitur:

$$p(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Estimasi parameter:

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_j^{(i)}, \qquad \sigma_j^2 = \frac{1}{m} \sum_{i=1}^m \left(x_j^{(i)} - \mu_j\right)^2.$$

Model banyak fitur independen:

$$p(x) = \prod_{j=1}^n p(x_j; \mu_j, \sigma_j^2).$$

Keputusan:

$$p(x) < \varepsilon \Rightarrow \text{anomaly}, \qquad p(x) \geq \varepsilon \Rightarrow \text{normal}.$$

Pilih ε menggunakan validation set dan metrik seperti F_1 .

Multivariate Gaussian untuk fitur saling berkorelasi:

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu)\right).$$

Learning with Large Datasets

Data besar dapat meningkatkan kinerja, tetapi training full batch mahal.

Metode	Inti
Batch GD	update memakai semua m data
Stochastic GD	update memakai 1 data setiap langkah
Mini-batch GD	update memakai b data setiap langkah

SGD update untuk satu contoh $(x^{(i)}, y^{(i)})$:

$$\theta_j := \theta_j - \alpha \left(h_{\theta}(x^{(i)}) - y^{(i)}\right) x_j^{(i)}.$$

Mini-batch update untuk batch B berisi b contoh:

$$\theta := \theta - \alpha \frac{1}{b} \sum_{i \in B} \left(h_{\theta}(x^{(i)}) - y^{(i)}\right) x^{(i)}.$$

Tips: lakukan error analysis sebelum menambah data, pastikan fitur informatif, dan plot learning curve untuk melihat apakah data tambahan membantu.

Q-Learning dan Q-Table

Q-Learning adalah algoritma RL untuk mempelajari nilai aksi $Q(s, a)$, yaitu kualitas memilih action a saat berada di state s .

$$Q(s, a) \approx \mathbb{E}[\text{future reward} \mid s, a].$$

Update Q-Learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\right].$$

Bentuk ekuivalen:

$$Q_{new}(s, a) = (1 - \alpha)Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a')\right].$$

TD target:

$$\text{TD target} = r + \gamma \max_{a'} Q(s', a').$$

TD error:

$$\delta = r + \gamma \max_{a'} Q(s', a') - Q(s, a).$$

Exploration vs exploitation: exploration mencoba aksi baru; exploitation memilih aksi terbaik saat ini. Epsilon-greedy:

$$a = \begin{cases} \text{aksi acak,} & \text{dengan peluang } \varepsilon, \\ \arg \max_a Q(s, a), & \text{dengan peluang } 1 - \varepsilon. \end{cases}$$

Q-table: tabel nilai $Q(s, a)$. Baris = state, kolom = action, isi = nilai Q. Agent memilih action bernilai Q terbesar pada state saat ini.

State	Left	Right	Up	Down
Start	0.10	0.80	0.20	-0.10
Middle	0.30	1.40	0.50	0.00
NearGoal	0.60	2.50	1.20	0.40
Trap	-1.00	-0.50	0.10	-2.00

Contoh: di state **Middle**, action terbaik adalah **Right** karena $Q(\text{Middle}, \text{Right}) = 1.40$ paling besar.

Contoh update satu langkah:

$$Q(s, a) = 0, \quad r = 10, \quad \gamma = 0.9, \quad \max_{a'} Q(s', a') = 5, \quad \alpha = 0.1.$$

$$Q_{new} = 0 + 0.1 [10 + 0.9(5) - 0] = 1.45.$$

Istilah Penting RL

Agent: pembelajar/pengambil keputusan. **Environment:** dunia tempat agent bertindak. **State:** kondisi saat ini. **Action:** pilihan tindakan. **Reward:** sinyal nilai baik/buruk.

Policy: strategi memilih aksi.

$$\pi(a \mid s) = P(A_t = a \mid S_t = s).$$

Value function: nilai state.

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s].$$

Action-value function: nilai state-action.

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Discount factor γ : dekat 0 fokus reward cepat; dekat 1 mempertimbangkan reward jangka panjang. **Episode:** satu sesi dari start sampai terminal state.

Ringkasan Pemilihan Algoritma

Masalah	Algoritma umum
Output angka kontinu	Linear regression, polynomial regression, NN regression
Output kelas biner	Logistic regression, SVM, neural network
Output banyak kelas	One-vs-all, softmax/NN multi-output, SVM multi-class
Data tanpa label	K-Means, PCA
Data aneh/jarang	Anomaly detection
Gambar	CNN
Fitur sangat banyak	Regularization, PCA, SVM kernel, NN
Data sangat besar	SGD/mini-batch, vectorization
Keputusan berurutan	Reinforcement learning, Q-learning

Kata Kunci Cepat

Istilah	Ingat Cepat
Linearly separable	Bisa dipisahkan garis/hyperplane.
Decision boundary	Batas pemisah kelas.
Margin	Jarak boundary ke data terdekat pada SVM.
Kernel	Fungsi similarity untuk boundary non-linear.
Centroid	Pusat cluster pada K-Means.
Eigen/SVD	Dasar PCA mencari arah variance terbesar.
Threshold	Batas keputusan, misal 0.5 atau ε .
One-hot vector	Label kelas sebagai vektor dengan satu posisi bernilai 1.
Bias unit	Input tetap 1 agar model punya intercept.
Cost/Loss	Ukuran kesalahan yang diminimalkan.
Gradient	Arah kenaikan cost; GD bergerak berlawanan arah.
Generalization	Kemampuan bekerja pada data baru.
Q-value	Nilai kualitas action pada state tertentu.

Kesalahan Umum Saat Ujian

- Mengira logistic regression untuk regresi kontinu. Logistic regression adalah untuk klasifikasi.
- Melupakan $x_0 = 1$ dan θ_0 sebagai bias.
- Mengregularisasi θ_0 , padahal biasanya tidak.
- Salah membaca confusion matrix: FP = aktual 0 tetapi diprediksi 1; FN = aktual 1 tetapi diprediksi 0.
- Menganggap accuracy selalu cukup; pada data tidak seimbang, periksa precision, recall, dan F_1 .
- Pada PCA, memakai label y . PCA tidak memakai label karena unsupervised.
- Pada anomaly detection, lupa bahwa anomaly jika $p(x) < \varepsilon$.
- Pada K-Means, lupa mengulang random initialization.
- Pada Q-learning, lupa memakai nilai state berikutnya $\max_{a'} Q(s', a')$.
- Mencampur learning rate α , regularization λ , discount factor γ , dan exploration rate ε .

Pola Jawaban Singkat

Perceptron vs ADALINE: Perceptron update berdasarkan output threshold/salah-benar klasifikasi; ADALINE memakai output linear dan error kontinu sehingga dapat diminimasi dengan MSE/LMS.

Supervised vs unsupervised vs RL: supervised punya label, unsupervised tanpa label, RL belajar dari reward setelah agent bertindak.

Q-table: tabel $Q(s, a)$ yang menyimpan estimasi nilai setiap action pada setiap state. Pilih action dengan nilai terbesar, kecuali sedang exploration.

Inti Q-learning: update nilai aksi lama menuju target baru $r + \gamma \max_{a'} Q(s', a')$ menggunakan learning rate α .

Overfitting: training bagus tetapi test buruk; solusinya regularisasi, tambah data, kurangi fitur/model, atau validasi model.

Underfitting: training dan test sama-sama buruk; solusinya tambah fitur, model lebih kompleks, kurangi regularisasi, atau training lebih baik.